

ESCS201 — Programming for Problem Solving

Solved Previous Year Questions (2018–19, 2022–23, 2023–24, 2024–25)

Module-wise Model Answers for Quick Exam Revision

MAKAUT B.Tech — Even Semester 2

Contents

1	Module 1 — Introduction to Programming	2
1.1	Group A — One-mark answers	2
1.2	Compiler vs Interpreter [2023–24, 5 marks; also in 2022–23 Group C]	3
1.3	Preprocessor Directives [2024–25, 5 marks; 2018–19, 2+3]	3
1.4	Types of Computers [2023–24, 15 marks]	3
1.5	OS, Functions of OS, Booting, Compiler vs Interpreter [2022–23, 4+4+3+4]	4
1.6	EOF, File Modes, fseek/ftell/rewind, Merging Two Sorted Files [2018–19, 2+2+3+8]	4
1.7	Short Notes [2018–19, 3×5 marks]	5
1.8	Dynamic Memory Allocation; malloc vs calloc [2018–19, 2+3]	5
2	Module 2 — Arithmetic Expressions and Precedence	5
3	Module 3 — Conditional Branching and Loops	6
3.1	Group A — One-mark answers	6
3.2	switch vs nested-if [2024–25, 5 marks]	7
3.3	break vs continue [2023–24; 2018–19 (with example), 5 marks]	7
3.4	Calculator using switch-case [2023–24, 5 marks]	7
3.5	Conditional (Ternary) Operator [2022–23, 5 marks]	7
3.6	Bitwise Left Shift Program [2024–25, 5 marks]	8
3.7	while vs do-while [2018–19, 5 marks]	8
3.8	Recursion vs Iteration [2018–19, 5 marks]	8
3.9	Primes in a Range + Vowel/Consonant Check [2024–25, 8+7]	8
3.10	Even/Odd, Prime Check, Sum of Even Elements [2022–23, 5+5+5]	9
3.11	Right-angled Star Pattern [2018–19, 10 marks; also Module 4]	9
4	Module 4 — Arrays	10
4.1	Group A — One-mark answers	10
4.2	Count Even and Odd Numbers in a 1-D Array [2024–25, 5 marks]	10
4.3	2-D Array Address Calculation [2018–19, 1+4 marks]	10
4.4	Max/Min via Function + Palindrome Numbers in Range [2023–24, 7+8]	11
4.5	Diagonal Matrix, String Concatenation, Sorting [2022–23, 5+5+5]	11
4.6	Series $1/4! - 1/6! + 1/8! - \dots \pm 1/n!$ [2018–19, 8 marks]	12
5	Module 5 — Basic Algorithms	12
5.1	Group A — One-mark answers	12
5.2	Asymptotic Notations [2023–24, 15 marks]	13
5.3	Merge Sort Algorithm + Complexity [2023–24, 10+5]	13
5.4	Number Systems [2022–23, 4+4+3+4]	14

6	Module 6 — Functions	14
6.1	Group A — One-mark answers	14
6.2	Call by Value vs Call by Reference [2023–24, 5 marks]	15
6.3	Area and Circumference of a Circle using Functions [2024–25, 5 marks]	15
6.4	Tracing Fun(2,3) and Fun(14,3) [2018–19, 5 marks]	15
6.5	Factorial and Fibonacci [2023–24, 8+7]	15
6.6	String Length without strlen() [2018–19, 7 marks]	16
7	Module 7 — Recursion	16
7.1	Group A — One-mark answers	16
7.2	Recursive Factorial [2023–24, 5 marks]	16
7.3	Flowchart for Factorial [2024–25 & 2022–23, 5 marks]	17
7.4	Fibonacci using Loop [2024–25, 5 marks]	17
7.5	Sum $S = 1! + 2! + \dots + n!$ using Recursive Factorial [2024–25, 8 marks]	17
7.6	Recursive GCD + Sorting Names [2024–25, 5+10]	17
8	Module 8 — Structures	18
8.1	Group A — One-mark answers	18
8.2	Structure and Array of Structures [2022–23, 5 marks]	18
8.3	Array vs Structure, Structure vs Union, Student Records Program [2022–23, 4+3+8]	18
8.4	Strings, Memory Representation, Palindrome String [2024–25, 5+4+6]	19
9	Module 9 — Pointers	20
9.1	Group A — One-mark answers	20
9.2	Length of an Array using Pointer Arithmetic [2024–25, 5 marks]	20
9.3	Armstrong Number [2018–19, 5 marks]	20
9.4	Output Prediction with Reasons [2018–19, 2.5+2.5]	20
10	Last-Minute Revision Sheet	21

1 Module 1 — Introduction to Programming

1.1 Group A — One-mark answers

2024–25 *Why do we use header files?*

Ans. Header files (`.h`) contain **declarations** of library functions, macros and constants (e.g. `printf()` in `stdio.h`). Including them lets the compiler check that we call these functions correctly.

2023–24 *Which language is written in binary codes only?*

Ans. **Machine language** (0s and 1s) — the only language the CPU executes directly.

2023–24 *The C preprocessors are specified with which symbol?*

Ans. The hash symbol `#` (e.g. `#include`, `#define`).

2022–23 *What are preprocessors?*

Ans. The preprocessor is a program that processes the source code **before compilation** — it expands `#include` files, replaces macros (`#define`) and handles conditional compilation (`#ifdef`).

2022–23 *What is the use of / What is the goto statement in C?*

Ans. `goto` is an **unconditional jump** statement that transfers control to a labelled statement within the same function: `goto label; ...label: stmt;`. Its use is discouraged because it destroys structured flow.

2022–23 *What is a Macro?*

Ans. A macro is a fragment of code given a name using `#define`; the preprocessor replaces every occurrence of the name by the code by **textual substitution**. E.g. `#define PI 3.14159` or `#define SQR(x) ((x)*(x))`.

2022–23 *What header file is used for string operations?*

Ans. `string.h` (for `strlen`, `strcpy`, `strcat`, `strcmp`, ...).

2018–19 *Which is true about `int *f()` and `int (*f)()`?*

Ans. (c) `int *f()` is a function returning a pointer to `int`; `int (*f)()` is a pointer to a function returning `int`.

2018–19 *The controlling expression of a switch cannot be of type:*

Ans. (c) `float` — switch works only on integral types (`int`, `char`, `enum`).

2018–19 *Operating system is a:*

Ans. (b) **System software**.

2018–19 *ALU is a part of:*

Ans. (c) **CPU** (CPU = ALU + Control Unit + Registers).

2018–19 *The value of EOF is:*

Ans. (a) `-1` (defined in `stdio.h`).

1.2 Compiler vs Interpreter

[2023–24, 5 marks; also in 2022–23 Group C]

Ans.

	Compiler	Interpreter
Translation	Whole program at once into machine code	Line by line, executing each line
Output	Produces an object/executable file	No object file; executes directly
Speed of execution	Fast (translation done once)	Slow (translates every run)
Error reporting	All errors listed after full scan	Stops at the first error in a line
Memory	Needs more memory (object code)	Needs less memory
Examples	C, C++ (gcc)	Python, BASIC, MATLAB

1.3 Preprocessor Directives

[2024–25, 5 marks; 2018–19, 2+3]

Ans. Preprocessor directives are commands beginning with # that are executed **before compilation**. Main directives:

- `#include` — inserts the contents of a header file: `#include <stdio.h>`
- `#define` — defines macros/symbolic constants: `#define MAX 100`
- `#undef` — removes a macro definition.
- `#ifdef` / `#ifndef` / `#endif` — conditional compilation.

```
#include <stdio.h>
#define PI 3.14159
#define AREA(r) (PI*(r)*(r)) /* macro with argument */
int main() { printf("%f", AREA(2.0)); return 0; }
```

Macro vs Function: a macro is expanded by *textual substitution* at preprocessing time (no function-call overhead, no type checking, code size increases); a function is compiled once and *called* at run time (call overhead, but type-checked and memory-efficient).

1.4 Types of Computers

[2023–24, 15 marks]

Ans. Computers are classified (A) by purpose, (B) by signal type, (C) by size and capability.

(A) **By purpose:** general-purpose (PCs) and special-purpose (washing-machine controller, ATM).

(B) **By signal:**

1. *Analog* — work on continuous signals (speedometer, thermometer).
2. *Digital* — work on discrete binary data; all modern computers.
3. *Hybrid* — combine both (ICU patient monitors, fuel pumps).

(C) **By size and capability:**

1. **Supercomputer** — fastest, costliest; used for weather forecasting, nuclear simulation; e.g. PARAM, Cray.
2. **Mainframe** — very large, supports thousands of simultaneous users; banking, railways reservation; e.g. IBM zSeries.
3. **Minicomputer** — mid-sized multi-user machine for departments/labs; e.g. PDP-11.
4. **Microcomputer (PC)** — single-user desktop/laptop built around a microprocessor.

5. **Workstation** — high-end single-user machine for CAD, graphics, research.

6. **Portable/embedded** — laptops, tablets, smartphones, embedded controllers.

Conclusion: from supercomputers to embedded chips, all share the same von Neumann organisation (Input → CPU(ALU+CU)+Memory → Output) but differ in speed, size, cost and number of users.

1.5 OS, Functions of OS, Booting, Compiler vs Interpreter [2022–23, 4+4+3+4]

(a) **Ans.** An **Operating System** is system software that acts as an *interface between the user and the hardware* and manages all resources of the computer (CPU, memory, files, devices). Examples: Windows, Linux, macOS, Android.

(b) **Functions of an OS:**

1. **Process management** — creating, scheduling and terminating processes (CPU scheduling).

2. **Memory management** — allocating/deallocating RAM to programs.

3. **File management** — organising data into files and directories; access control.

4. **Device management** — controlling I/O devices through drivers; spooling.

5. **Security & user interface** — login/passwords, protection, and providing CLI/GUI.

(c) **Boot**ing is the process of **starting the computer and loading the operating system** from disk into main memory. When power is switched on, the ROM-BIOS runs the POST (self-test) and then the bootstrap loader copies the OS kernel into RAM and hands over control. *Cold booting* = starting from power-off; *warm booting* = restarting without switching power off.

(d) Compiler vs Interpreter — see table in §1.2.

1.6 EOF, File Modes, fseek/ftell/rewind, Merging Two Sorted Files [2018–19, 2+2+3+8]

(a) **Ans.** EOF (End Of File) is a constant (−1) defined in `stdio.h`, returned by input functions like `getc()` when no more data can be read — it is the standard way to detect the end of a file or input stream.

(b) "a" opens a file for **appending only** — writing happens at the end; reading is *not* allowed. "a+" opens for **appending and reading** — the file can be read anywhere, but all writes still go to the end. Both create the file if it does not exist.

(c)

- `fseek(fp, offset, origin)` — moves the file pointer `offset` bytes from `SEEK_SET` (beginning), `SEEK_CUR` (current) or `SEEK_END` (end). Enables random access.
- `ftell(fp)` — returns the current position (in bytes) of the file pointer.
- `rewind(fp)` — resets the file pointer to the beginning of the file.

(d) Merge two sorted files (command-line arguments):

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    FILE *f1, *f2, *f3;    int a, b, r1, r2;
    if (argc != 4) { printf("Usage: %s in1 in2 out\n", argv[0]); return 1; }
    f1 = fopen(argv[1], "r"); f2 = fopen(argv[2], "r");
    f3 = fopen(argv[3], "w");
    if (!f1 || !f2 || !f3) { printf("File error\n"); return 1; }
```

```

    r1 = fscanf(f1, "%d", &a);  r2 = fscanf(f2, "%d", &b);
    while (r1 == 1 && r2 == 1) {
        if (a <= b) { fprintf(f3, "%d ", a); r1 = fscanf(f1, "%d", &a); }
        else      { fprintf(f3, "%d ", b); r2 = fscanf(f2, "%d", &b); }
    }
    while (r1 == 1) { fprintf(f3, "%d ", a); r1 = fscanf(f1, "%d", &a); }
    while (r2 == 1) { fprintf(f3, "%d ", b); r2 = fscanf(f2, "%d", &b); }
    fclose(f1); fclose(f2); fclose(f3);
    return 0;
}

```

1.7 Short Notes

[2018–19, 3×5 marks]

(a) **Compiler.** A compiler translates the entire high-level source program into machine code in one pass, producing an object file. Phases: lexical analysis → syntax analysis → semantic analysis → code optimisation → code generation. It reports all errors together and the compiled code runs fast. Example: `gcc` for C.

(b) **Type Casting.** Converting a value from one data type to another. *Implicit* conversion is done automatically by the compiler (`int` promoted to `float` in mixed expressions); *explicit* casting is forced by the programmer: `(type)expression`.

```

int a = 7, b = 2;
float x = (float)a / b;    /* 3.5, not 3 */

```

(c) **Pointer to Pointer.** A variable that stores the address of another pointer; declared with `**`. Used in dynamic 2-D arrays and to modify a pointer inside a function.

```

int a = 5; int *p = &a; int **q = &p;
printf("%d", **q);    /* 5 */

```

(d) **Operating System.** See §1.5(a)–(b): system software that manages hardware resources (CPU, memory, files, devices) and provides the user interface; e.g. Windows, Linux.

(e) **Function Prototype.** A declaration that tells the compiler a function's *name*, *return type* and *parameter types* before it is used, enabling type checking of calls: `int add(int, int);`. The definition may then appear after `main()`.

1.8 Dynamic Memory Allocation; `malloc` vs `calloc`

[2018–19, 2+3]

Ans. Dynamic memory allocation means allocating memory at *run time* from the heap using library functions (`malloc`, `calloc`, `realloc`, `free` in `stdlib.h`), so the program can request exactly as much memory as it needs while running.

	<code>malloc()</code>	<code>calloc()</code>
Syntax	<code>malloc(n*size)</code> — 1 argument	<code>calloc(n, size)</code> — 2 arguments
Initialisation	Memory contains <i>garbage</i>	Memory initialised to <i>zero</i>
Speed	Slightly faster	Slower (zero-fills)

```

int *p = (int *)malloc(5 * sizeof(int));
int *q = (int *)calloc(5, sizeof(int));    /* all zeros */
free(p); free(q);

```

2 Module 2 — Arithmetic Expressions and Precedence

2018–19 Value of $x = (3^6) + (2^2)$?

Ans. (b) 5. In C, `^` is bitwise **XOR**, not power. $3 \oplus 6 = 011 \oplus 110 = 101 = 5$; $2 \oplus 2 = 0$; so

$x = 5 + 0 = 5$.

2023–24 Output of: `int a=3, *b=&a; printf("%d", a*b);`

Ans. Compilation error. `b` is a pointer; multiplying an `int` by a pointer (`a*b`) is an invalid operation. (`a * (*b)` would print 9.)

2024–25 Output of: `int i=3; i=i++; printf("%d",i);`

Ans. 3 (on common compilers). `i++` returns the old value 3 which is assigned back to `i`. Strictly, modifying `i` twice between sequence points is **undefined behaviour** — worth mentioning for full marks.

2022–23 If `a=2`, value of `printf("%d",-a);`?

Ans. 1. Pre-decrement first decreases `a` to 1, then uses it.

Quick-revision tip: Precedence ladder (high $\rightarrow$ low): `()` > unary `++ -- ! & * > * / % > + - > << >>` > relational `> == != > & > ^ > | > && > || > ?: > =`. Note: `^` is XOR, C has *no* power operator (use `pow()`).

3 Module 3 — Conditional Branching and Loops

3.1 Group A — One-mark answers

2024–25 Output of `for(int i=0; i<=5; i++); { printf("%d", i); }`

Ans. Compilation error. The semicolon after `for(...)` ends the (empty) loop body; the block then tries to print `i`, but `i` was declared *inside the for* and is **out of scope**. (If `i` were declared outside, the output would be 6.)

2023–24 Switch on a `double ch` (value 1 entered)?

Ans. Compilation error — the controlling expression of `switch` must be of *integral* type; `double` is not allowed.

2022–23 Use of `continue` in a loop?

Ans. continue skips the remaining statements of the current iteration and jumps to the next iteration (loop condition/update) without leaving the loop.

2022–23 What is nested `if-else`?

Ans. An `if` or `else` that contains *another* `if-else` inside it, used to test multiple conditions hierarchically: `if (a>b) { if (a>c) ... } else ...`

2018–19 `enum Months {JAN=10,FEB,MAR,APR}; Months X=FEB; switch(X)...`

Ans. (c) Generates compilation error — in C, the type must be written `enum Months X`; (using `Months` alone is illegal without a `typedef`). (Had it compiled, `FEB = 11` would match *case 11* and print “February”.)

2018–19 Output of `for(i=0;i<5;i++) printf("%d\n", 1L<<i);`

Ans. (d) 1, 2, 4, 8, 16 — each left shift multiplies by 2: $2^0 \dots 2^4$.

3.2 switch vs nested-if

[2024–25, 5 marks]

Ans.

	switch	nested if-else
Condition type	Tests <i>equality</i> of one integral expression against constant cases	Can test <i>any</i> condition: ranges, floats, logical combinations
Data types	int, char, enum only	All data types
Readability/speed	Cleaner and often faster (jump table) for many fixed choices	Becomes deep and hard to read
Fall-through	Needs break after each case	Not applicable

```
switch (grade) {                                /* menu-style choice */
    case 'A': printf("Excellent"); break;
    case 'B': printf("Good"); break;
    default : printf("Try again");
}
if (m >= 80) printf("A");                        /* range test: needs if-else */
else if (m >= 60) printf("B");
else printf("C");
```

3.3 break vs continue

[2023–24; 2018–19 (with example), 5 marks]

Ans. **break** terminates the loop (or switch) entirely; control jumps to the first statement after the loop. **continue** only abandons the current iteration and continues with the next one.

```
for (i = 1; i <= 5; i++) {
    if (i == 3) break;           /* prints 1 2 */
    printf("%d ", i);
}
for (i = 1; i <= 5; i++) {
    if (i == 3) continue;       /* prints 1 2 4 5 */
    printf("%d ", i);
}
```

3.4 Calculator using switch-case

[2023–24, 5 marks]

```
#include <stdio.h>
int main() {
    float a, b; char op;
    printf("Enter num1 operator num2 (e.g. 4 * 5): ");
    scanf("%f %c %f", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f", a + b); break;
        case '-': printf("%.2f", a - b); break;
        case '*': printf("%.2f", a * b); break;
        case '/': if (b != 0) printf("%.2f", a / b);
                   else printf("Division by zero!"); break;
        default : printf("Invalid operator");
    }
    return 0;
}
```

3.5 Conditional (Ternary) Operator

[2022–23, 5 marks]

Ans. The conditional operator `?:` is the only **ternary** operator in C. Syntax: `condition ? expr1 : expr2` — if the condition is true the whole expression takes the value of `expr1`, otherwise `expr2`. It is a compact one-line replacement for simple if-else.

```
big = (a > b) ? a : b;                      /* larger of two */
```



```
max = (a>b) ? ((a>c)?a:c) : ((b>c)?b:c); /* largest of three (nested) */
printf("%s", (n%2==0) ? "Even" : "Odd");
```

3.6 Bitwise Left Shift Program

[2024–25, 5 marks]

```
#include <stdio.h>
int main() {
    int n, s;
    printf("Enter number and shift count: ");
    scanf("%d %d", &n, &s);
    printf("%d << %d = %d\n", n, s, n << s);
    return 0;
}
/* Example: n=5 (101), s=2 -> output: 5 << 2 = 20 (10100)
   Left shift by s multiplies the number by 2^s. */
```

3.7 while vs do-while

[2018–19, 5 marks]

Ans. **while** is an **entry-controlled** loop — the condition is tested *before* the body, so the body may execute **zero** times. **do-while** is **exit-controlled** — the body executes first and the condition is tested after, so the body runs **at least once**; it ends with a semicolon.

```
int i = 10;
while (i < 5) printf("A"); /* prints nothing */
do { printf("B"); } while (i < 5); /* prints B once */
```

Use **do-while** for menus where the choice must be read at least once.

3.8 Recursion vs Iteration

[2018–19, 5 marks]

Ans. **Recursion** solves a problem by a function calling itself with a smaller input until a *base case*; **iteration** repeats a block using loop constructs.

	Recursion	Iteration
Mechanism	Function calls itself; uses the <i>stack</i>	Loop (for/while); uses a counter
Termination	Base case	Loop condition fails
Memory/speed	More memory (stack frames), slower	Less memory, faster
Code	Short, elegant for trees, Fibonacci, Tower of Hanoi	Better for simple repetition

```
int factR(int n){ return (n<=1) ? 1 : n*factR(n-1); } /* recursion */
int factI(int n){ int f=1,i; for(i=2;i<=n;i++) f*=i; return f; }
```

3.9 Primes in a Range + Vowel/Consonant Check

[2024–25, 8+7]

(a)

```
#include <stdio.h>
int main() {
    int lo, hi, n, i, flag;
    printf("Enter range: "); scanf("%d %d", &lo, &hi);
    for (n = lo; n <= hi; n++) {
        if (n < 2) continue;
        flag = 1;
        for (i = 2; i * i <= n; i++)
            if (n % i == 0) { flag = 0; break; }
        if (flag) printf("%d ", n);
    }
    return 0;
}
```

```
}
```

(b)

```
#include <stdio.h>
int main() {
    char ch;
    printf("Enter a character: "); scanf(" %c", &ch);
    if ((ch>='a'&&ch<='z') || (ch>='A'&&ch<='Z')) {
        switch (ch) {
            case 'a': case 'e': case 'i': case 'o': case 'u':
            case 'A': case 'E': case 'I': case 'O': case 'U':
                printf("Vowel"); break;
            default: printf("Consonant");
        }
    } else printf("wrong input");
    return 0;
}
```

3.10 Even/Odd, Prime Check, Sum of Even Elements

[2022–23, 5+5+5]

```
/* (a) even or odd */
#include <stdio.h>
int main(){ int n; scanf("%d",&n);
    printf((n%2==0)?"Even":"Odd"); return 0; }
```

```
/* (b) prime or not */
#include <stdio.h>
int main(){ int n,i,flag=1; scanf("%d",&n);
    if(n<2) flag=0;
    for(i=2;i*i<=n;i++) if(n%i==0){flag=0;break;}
    printf(flag?"Prime":"Not Prime"); return 0; }
```

```
/* (c) sum of even numbers in an array */
#include <stdio.h>
int main(){ int a[100],n,i,sum=0;
    printf("n? "); scanf("%d",&n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    for(i=0;i<n;i++) if(a[i]%2==0) sum+=a[i];
    printf("Sum of even = %d",sum); return 0; }
```

3.11 Right-angled Star Pattern

[2018–19, 10 marks; also Module 4]

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Rows? "); scanf("%d", &n);
    for (i = 1; i <= n; i++) {           /* row loop */
        for (j = 1; j <= i; j++)         /* star loop */
            printf("* ");
        printf("\n");
    }
    return 0;
}

/* n=4:
*
* *
* * *
* * * * */
```

(For a centred pyramid, first print $n-i$ spaces, then $2i-1$ stars.)

4 Module 4 — Arrays

4.1 Group A — One-mark answers

2023–24 *Output: `int a[2][3] = {1,2,3,4,5};` printed element by element?*

Ans. 123450 — missing initialisers are set to 0, so the array is $\{\{1,2,3\},\{4,5,0\}\}$.

2023–24 *Assigning to an element whose subscript exceeds the array size?*

Ans. C performs **no bounds checking**: the program compiles and writes past the array into adjacent memory — **undefined behaviour** (garbage results or a crash/segmentation fault).

2024–25 *Output: `char x='a'; printf("%d\n", x);`*

Ans. 97 — %d prints the ASCII value of 'a'.

2022–23 *if ((char)a[i] == '5')... over array {1,2,3,4,5}?*

Ans. Prints **FAIL five times**. '5' is the character with ASCII value 53; the array elements are integers 1–5, never 53, so the condition is always false.

4.2 Count Even and Odd Numbers in a 1-D Array

[2024–25, 5 marks]

```
#include <stdio.h>
int main() {
    int a[100], n, i, even = 0, odd = 0;
    printf("n? "); scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    for (i = 0; i < n; i++)
        (a[i] % 2 == 0) ? even++ : odd++;
    printf("Even = %d, Odd = %d", even, odd);
    return 0;
}
```

4.3 2-D Array Address Calculation

[2018–19, 1+4 marks]

Ans. An **array** is a collection of elements of the same data type stored in contiguous memory locations and accessed by a common name and index.

Given `DATA[15][20]`, word size $w = 4$ bytes, base address $B = 3020$; find address of `DATA[12][13]` (0-based, rows $N_r = 15$, columns $N_c = 20$).

Row-major:

$$\text{Addr} = B + w(i \cdot N_c + j) = 3020 + 4(12 \times 20 + 13) = 3020 + 4(253) = \mathbf{4032}$$

Column-major:

$$\text{Addr} = B + w(j \cdot N_r + i) = 3020 + 4(13 \times 15 + 12) = 3020 + 4(207) = \mathbf{3848}$$

4.4 Max/Min via Function + Palindrome Numbers in Range [2023–24, 7+8]

```
/* (a) max and min using a function that returns an array */
#include <stdio.h>
int *maxmin(int a[], int n) {
    static int r[2]; int i;
    r[0] = r[1] = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > r[0]) r[0] = a[i];
        if (a[i] < r[1]) r[1] = a[i];
    }
    return r;          /* static array survives return */
}
int main() {
    int a[100], n, i, *p;
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    p = maxmin(a, n);
    printf("Max = %d, Min = %d", p[0], p[1]);
    return 0;
}
```

```
/* (b) palindrome numbers in a range using a function */
#include <stdio.h>
int isPalin(int n) {
    int rev = 0, t = n;
    while (t) { rev = rev*10 + t%10; t /= 10; }
    return rev == n;
}
int main() {
    int lo, hi, n;
    scanf("%d %d", &lo, &hi);
    for (n = lo; n <= hi; n++)
        if (isPalin(n)) printf("%d ", n);
    return 0;
}
```

4.5 Diagonal Matrix, String Concatenation, Sorting [2022–23, 5+5+5]

```
/* (a) print diagonal of a 5x5 matrix */
#include <stdio.h>
int main() {
    int a[5][5], i, j;
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++) scanf("%d", &a[i][j]);
    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++)
            (i == j) ? printf("%4d", a[i][j]) : printf("%4d", 0);
        printf("\n");
    }
    return 0;
}
```

```
/* (b) concatenate two strings without strcat() */
#include <stdio.h>
int main() {
    char s1[100], s2[50]; int i = 0, j = 0;
    scanf("%s %s", s1, s2);
    while (s1[i] != '\0') i++;          /* go to end of s1 */
    while (s2[j] != '\0') s1[i++] = s2[j++];
}
```

```

    s1[i] = '\0';
    printf("%s", s1);
    return 0;
}

```

```

/* (c) sort n elements (bubble sort) */
#include <stdio.h>
int main() {
    int a[100], n, i, j, t;
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    for (i = 0; i < n-1; i++)
        for (j = 0; j < n-1-i; j++)
            if (a[j] > a[j+1]) { t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    return 0;
}

```

4.6 Series $1/4! - 1/6! + 1/8! - \dots \pm 1/n!$

[2018–19, 8 marks]

```

#include <stdio.h>
int main() {
    int n, i, k, sign = 1;
    double sum = 0, fact;
    printf("Enter n (even, >=4): "); scanf("%d", &n);
    for (i = 4; i <= n; i += 2) {
        fact = 1;
        for (k = 2; k <= i; k++) fact *= k;
        sum += sign * (1.0 / fact);
        sign = -sign; /* alternate the sign */
    }
    printf("Sum = %lf", sum);
    return 0;
}

```

5 Module 5 — Basic Algorithms

5.1 Group A — One-mark answers

2023–24 / 2022–23 *Worst-case / time complexity of bubble sort?*

Ans. $O(n^2)$ (worst and average case); best case $O(n)$ with the early-exit flag.

2024–25 *Arrangement of playing cards is an example of which sorting technique?*

Ans. **Insertion sort** — each new card is inserted into its proper place among the already-sorted cards.

2023–24 *Advantage of selection sort?*

Ans. It performs the **minimum number of swaps** — at most $(n - 1)$ — so it is useful when write/swap operations are costly. It is also simple and in-place.

2022–23 *What is linear searching in an array?*

Ans. Checking every element **one by one from the first to the last** until the key is found or the array ends. Works on unsorted arrays; complexity $O(n)$.

2018–19 Which is a bitwise operator?

Ans. (b) $\ll$ and (c) $|$ are bitwise (left shift, OR); $<$ is relational and $||$ is logical. (Standard key: (c) $|$ — but $\ll$ is also bitwise; mention both.)

5.2 Asymptotic Notations

[2023–24, 15 marks]

Ans. Asymptotic notations describe how an algorithm's running time grows with input size n , ignoring constants and lower-order terms.

1. **Big-O (O) — upper bound (worst case).** $f(n) = O(g(n))$ if there exist constants $c > 0$, n_0 such that $f(n) \leq c g(n)$ for all $n \geq n_0$.

Example: $3n^2 + 5n = O(n^2)$. Bubble sort is $O(n^2)$; binary search $O(\log n)$.

2. **Big-Omega (Ω) — lower bound (best case).** $f(n) = \Omega(g(n))$ if $f(n) \geq c g(n)$ for all $n \geq n_0$.

Example: any comparison sort is $\Omega(n \log n)$; linear search best case $\Omega(1)$.

3. **Big-Theta (Θ) — tight bound (exact order).** $f(n) = \Theta(g(n))$ if $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$, i.e. both O and Ω .

Example: merge sort is $\Theta(n \log n)$ in all cases.

4. **Little-o (o) — strict upper bound:** f grows *strictly slower* than g ; $n = o(n^2)$.

5. **Little-omega (ω) — strict lower bound:** f grows *strictly faster* than g ; $n^2 = \omega(n)$.

Common growth rates: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.

5.3 Merge Sort Algorithm + Complexity

[2023–24, 10+5]

Ans. Idea (divide and conquer): divide the array into two halves, recursively sort each half, then *merge* the two sorted halves.

Algorithm:

1. MERGESORT(A, low, high)
2. if low < high:
3. $mid \leftarrow (low + high)/2$
4. MERGESORT(A, low, mid)
5. MERGESORT(A, mid+1, high)
6. MERGE(A, low, mid, high)

MERGE compares the front elements of the two sorted halves and repeatedly copies the smaller one into a temporary array, then copies leftovers and copies the temp array back.

```
void merge(int a[], int l, int m, int h) {
    int t[100], i = l, j = m + 1, k = 0;
    while (i <= m && j <= h)
        t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= m) t[k++] = a[i++];
    while (j <= h) t[k++] = a[j++];
    for (i = l, k = 0; i <= h; i++) a[i] = t[k++];
}

void mergesort(int a[], int l, int h) {
    if (l < h) {
        int m = (l + h) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, h);
    }
}
```

```

    merge(a, l, m, h);
}
}

```

(b) **Time complexity:** the recurrence is

$$T(n) = 2T(n/2) + O(n)$$

(two half-size subproblems + linear merging). The recursion tree has $\log_2 n$ levels, each costing $O(n)$, so

$$T(n) = \Theta(n \log n) \quad (\text{best, average and worst case}); \quad \text{space } O(n).$$

5.4 Number Systems

[2022–23, 4+4+3+4]

(a) $(1101.01)_2$ to decimal:

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 8 + 4 + 0 + 1 + 0 + 0.25 = (13.25)_{10}$$

(b) **2's complement** of a binary number is obtained by inverting every bit (1's complement) and adding 1. It represents negative numbers in computers, so subtraction can be done by addition and there is a unique zero.

Example: $+5 = 0101$; 1's comp = 1010; $+1 \Rightarrow$ 2's comp = 1011 = -5 (in 4 bits).

(c) $10101_2 - 00111_2$ using **2's complement** ($21 - 7$):

$$\begin{aligned} \text{2's comp of } 00111 &= 11000 + 1 = 11001 \\ 10101 + 11001 &= 1\ 01110 \quad (\text{discard end carry}) \\ \Rightarrow \mathbf{01110_2} &= \mathbf{14_{10}} \checkmark \end{aligned}$$

(d) $(5673)_{10}$ to octal — divide by 8, read remainders bottom-up:

$$\begin{aligned} 5673 \div 8 &= 709 \text{ r } 1; & 709 \div 8 &= 88 \text{ r } 5; & 88 \div 8 &= 11 \text{ r } 0; & 11 \div 8 &= 1 \text{ r } 3; & 1 \div 8 &= 0 \text{ r } 1 \\ \Rightarrow (5673)_{10} &= (\mathbf{13051})_8 \end{aligned}$$

6 Module 6 — Functions

6.1 Group A — One-mark answers

2023–24 *Default return type of a function definition?*

Ans. `int` (in old C, a function without a stated return type is assumed to return `int`).

2024–25 *Main difference between call by value and call by address?*

Ans. Call by *value* passes a **copy** — the actual argument cannot change; call by *address* passes the **address (pointer)** — the function can modify the original variable.

2024–25 *Why is scope of a variable necessary in a function?*

Ans. Scope confines a variable to the block/function where it is declared — this **prevents accidental modification and name conflicts**, saves memory, and makes functions independent and reusable.

2023–24 *Limit on the number of functions in a C program?*

Ans. **No fixed limit** — a C program may contain any number of functions (memory permitting); it must have exactly one `main()`.

2018–19 A function may contain:

Ans. (c) more than two return statements — any number is allowed; the first one executed terminates the function.

6.2 Call by Value vs Call by Reference

[2023–24, 5 marks]

Ans.

	Call by Value	Call by Reference (address)
What is passed	Copy of the value	Address of the variable
Effect on actual argument	Unchanged	Can be changed via pointer
Formal parameter	Ordinary variable	Pointer (int *p)
Safety / use	Safe; default in C	Used when the function must modify caller's data (e.g. swap, scanf)

```
void swapV(int a, int b){ int t=a; a=b; b=t; } /* no effect */
void swapR(int *a,int *b){ int t=*a; *a=*b; *b=t; } /* swaps! */
/* call: swapV(x,y); swapR(&x,&y); */
```

6.3 Area and Circumference of a Circle using Functions

[2024–25, 5 marks]

```
#include <stdio.h>
#define PI 3.14159
float area(float r) { return PI * r * r; }
float circum(float r) { return 2 * PI * r; }
int main() {
    float r;
    printf("Radius? "); scanf("%f", &r);
    printf("Area = %.2f\nCircumference = %.2f", area(r), circum(r));
    return 0;
}
```

6.4 Tracing Fun(2,3) and Fun(14,3)

[2018–19, 5 marks]

Ans. Definition: $Fun(a,b) = 0$ if $a < b$; otherwise $Fun(a,b) = Fun(a-b,b) + 1$.

Fun(2,3): $2 < 3 \Rightarrow$ returns 0 immediately.

Fun(14,3):

$$Fun(14,3) = Fun(11,3) + 1 = Fun(8,3) + 2 = Fun(5,3) + 3 = Fun(2,3) + 4 = 0 + 4 = 4$$

Observation: the function counts how many times b can be subtracted from a , i.e. it computes the integer quotient $\lfloor a/b \rfloor$ ($14/3 = 4$).

6.5 Factorial and Fibonacci

[2023–24, 8+7]

```
/* (a) factorial of a number */
#include <stdio.h>
long fact(int n) {
    long f = 1; int i;
    for (i = 2; i <= n; i++) f *= i;
    return f;
}
int main(){ int n; scanf("%d",&n);
    printf("%d! = %ld", n, fact(n)); return 0; }
```



```

/* (b) Fibonacci series upto n terms */
#include <stdio.h>
int main() {
    int n, i, a = 0, b = 1, c;
    printf("Terms? "); scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("%d ", a);
        c = a + b; a = b; b = c;
    }
    return 0;
}
/* n=7 -> 0 1 1 2 3 5 8 */

```

6.6 String Length without strlen()

[2018–19, 7 marks]

```

#include <stdio.h>
int mylen(char s[]) {
    int i = 0;
    while (s[i] != '\0') i++; /* count till null character */
    return i;
}
int main() {
    char str[100];
    printf("Enter a string: ");
    scanf("%[^\n]", str); /* reads spaces too */
    printf("Length = %d", mylen(str));
    return 0;
}

```

7 Module 7 — Recursion

7.1 Group A — One-mark answers

2024–25 Define recursion.

Ans. Recursion is a technique in which a **function calls itself**, directly or indirectly, reducing the problem each time until a **base case** stops the calls.

2024–25 Explain Ackermann function.

Ans. The Ackermann function $A(m, n)$ is a classic **deeply recursive** function: $A(0, n) = n + 1$; $A(m, 0) = A(m - 1, 1)$; $A(m, n) = A(m - 1, A(m, n - 1))$. It grows extremely fast and is used to test recursion depth; it is computable but not primitive-recursive.

2022–23 Recursion uses which data structure?

Ans. Stack — each call's return address and local variables are pushed onto the call stack and popped on return (LIFO).

2023–24 Output of the recursive code $f1(4)$ (where $f1$ computes $x * f1(x-1)$, base $x==1$ returns 1)?

Ans. 24 — it is recursive factorial: $4 \times 3 \times 2 \times 1 = 24$.

7.2 Recursive Factorial

[2023–24, 5 marks]

```
#include <stdio.h>
long fact(int n) {
    if (n <= 1) return 1;          /* base case */
    return n * fact(n - 1);        /* recursive case */
}
int main(){ int n; scanf("%d",&n);
    printf("%d! = %ld", n, fact(n)); return 0; }
/* fact(4) = 4*fact(3) = 4*3*fact(2) = 4*3*2*fact(1) = 24 */
```

7.3 Flowchart for Factorial

[2024–25 & 2022–23, 5 marks]

Ans. Describe/draw the following boxes connected by arrows:

1. (Oval) START
2. (Parallelogram) Read n
3. (Rectangle) $f \leftarrow 1, i \leftarrow 1$
4. (Diamond) Is $i \leq n$? — if **No** go to step 7
5. (Rectangle) $f \leftarrow f \times i$
6. (Rectangle) $i \leftarrow i + 1$; go back to step 4
7. (Parallelogram) Print f
8. (Oval) STOP

(The diamond creates the loop; the No-branch exits to the output box.)

7.4 Fibonacci using Loop

[2024–25, 5 marks]

Ans. Same program as Module 6 §6.5(b) — iterative Fibonacci with three variables a , b , c .

7.5 Sum $S = 1! + 2! + \dots + n!$ using Recursive Factorial

[2024–25, 8 marks]

```
#include <stdio.h>
long fact(int n) { return (n <= 1) ? 1 : n * fact(n - 1); }
int main() {
    int n, i; long S = 0;
    printf("Enter n: "); scanf("%d", &n);
    for (i = 1; i <= n; i++)
        S += fact(i);          /* add i! each time */
    printf("S = %ld", S);
    return 0;
}
/* n=4 : 1+2+6+24 = 33 */
```

7.6 Recursive GCD + Sorting Names

[2024–25, 5+10]

```
/* (a) recursive GCD (Euclid's algorithm) */
#include <stdio.h>
int gcd(int a, int b) {
    if (b == 0) return a;          /* base case */
    return gcd(b, a % b);
}
int main(){ int a,b; scanf("%d %d",&a,&b);
    printf("GCD = %d", gcd(a,b)); return 0; }
/* gcd(48,18) -> gcd(18,12) -> gcd(12,6) -> gcd(6,0) = 6 */
```

```

/* (b) sort names of students alphabetically */
#include <stdio.h>
#include <string.h>
int main() {
    char name[20][30], t[30]; int n, i, j;
    printf("How many students? "); scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%s", name[i]);
    for (i = 0; i < n-1; i++) /* bubble sort on strings */
        for (j = 0; j < n-1-i; j++)
            if (strcmp(name[j], name[j+1]) > 0) {
                strcpy(t, name[j]);
                strcpy(name[j], name[j+1]);
                strcpy(name[j+1], t);
            }
    printf("Sorted names:\n");
    for (i = 0; i < n; i++) printf("%s\n", name[i]);
    return 0;
}

```

8 Module 8 — Structures

8.1 Group A — One-mark answers

2024–25 Main difference between Structure and Union?

Ans. In a **structure** each member has its *own* memory (size = sum of members); in a **union** all members *share the same* memory (size = largest member), so only one member holds a valid value at a time.

2023–24 What is the size of a C structure?

Ans. At least the **sum of the sizes of all its members**, usually rounded up by *padding/alignment* bytes added by the compiler; obtained with `sizeof(struct tag)`.

2018–19 Bytes allocated for `struct A {short int a; char ch; float f;};`?

Ans. (c) **No memory is allocated** — this is only a structure *definition* (a template); memory is allocated only when a *variable* of the structure is declared. (Also note the deliberate typos ‘*strcut*’/‘*flat*’ would not compile.)

8.2 Structure and Array of Structures

[2022–23, 5 marks]

Ans. A **structure** is a user-defined data type that groups variables of *different* types under one name. An **array of structures** is a collection of many such records — ideal for storing data of many students/employees.

```

struct Student { int roll; char name[30]; float marks; };
struct Student s[60]; /* array of 60 records */
s[0].roll = 1; s[0].marks = 88.5; /* access with . operator */

```

8.3 Array vs Structure, Structure vs Union, Student Records Program [2022–23, 4+3+8]

(a) Array vs Structure:

	Array	Structure
Element types	All of the <i>same</i> type	Members may be of <i>different</i> types
Access	By index: <code>a[i]</code>	By member name: <code>s.roll</code>
Keyword	none	<code>struct</code>
Example	<code>int marks[5];</code>	<code>struct {int roll; float m;} s;</code>

(b) **Structure vs Union:** memory — structure allocates separate memory to every member; union overlaps all members in one block sized by the largest member. `sizeof` the struct below is ≈ 8 bytes, the union only 4:

```
struct S { int i; float f; };    /* 4+4 = 8 bytes */
union U { int i; float f; };    /* max(4,4) = 4 */
```

(c) **Program — 10 students with average marks:**

```
#include <stdio.h>
struct Student { char name[30]; int roll; float marks; };
int main() {
    struct Student s[10]; int i; float sum = 0;
    for (i = 0; i < 10; i++) {
        printf("Name roll marks of student %d: ", i + 1);
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
        sum += s[i].marks;
    }
    printf("\nName\tRoll\tMarks\n");
    for (i = 0; i < 10; i++)
        printf("%s\t%d\t%.2f\n", s[i].name, s[i].roll, s[i].marks);
    printf("Average marks = %.2f", sum / 10);
    return 0;
}
```

8.4 Strings, Memory Representation, Palindrome String [2024–25, 5+4+6]

(a) **Ans.** A string in C is a one-dimensional array of characters terminated by the **null character** `'\0'`. E.g. `char s[] = "HELLO"`; occupies 6 bytes (5 letters + `'\0'`). Handled with `string.h` functions (`strlen`, `strcpy`, `strcmp`, `strcat`).

(b) **Memory representation:** characters are stored in *contiguous* bytes, one ASCII code per byte, ending with `'\0'`:

| 'H' | 'E' | 'L' | 'L' | 'O' | '\0' | (addresses 2000–2005)

The array name is the address of the first character; the null byte tells library functions where the string ends.

(c) **Palindrome string check:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100]; int i, n, flag = 1;
    printf("Enter string: "); scanf("%s", s);
    n = strlen(s);
    for (i = 0; i < n / 2; i++)
        if (s[i] != s[n-1-i]) { flag = 0; break; }
    printf(flag ? "Palindrome" : "Not a palindrome");
    return 0;
}
/* madam -> Palindrome */
```

9 Module 9 — Pointers

9.1 Group A — One-mark answers

2024–25 What is pointer arithmetic in C?

Ans. Arithmetic performed on addresses: `p+1` moves the pointer ahead by `sizeof(type)` bytes (not 1 byte); valid operations are `p±n`, `p++`, `p--` and *subtracting two pointers* of the same array (gives number of elements between them).

2018–19 Output of `int i=5; printf("%d%d%d%d", ++i, i--, --i, i++);`

Ans. (a) 5555. Most compilers evaluate `printf` arguments *right to left*: `i++→5` (`i=6`), `--i→5` (`i=5`), `i--→5` (`i=4`), `++i→5` (`i=5`); printed left to right: 5 5 5 5. (Strictly undefined behaviour — state this for full credit.)

9.2 Length of an Array using Pointer Arithmetic

[2024–25, 5 marks]

```
#include <stdio.h>
int main() {
    int a[] = {10, 20, 30, 40, 50, 60};
    int *start = a;
    int *end = &a[sizeof(a)/sizeof(a[0]) - 1];
    int len = (end - start) + 1;    /* pointer subtraction */
    printf("Length of array = %d", len);
    return 0;
}
/* end - start = 5 elements apart, +1 => 6 */
```

Key idea: subtracting two pointers of the same array yields the count of elements between them, not bytes.

9.3 Armstrong Number

[2018–19, 5 marks]

Ans. An Armstrong number equals the sum of its digits each raised to the power of the number of digits ($153 = 1^3 + 5^3 + 3^3$).

```
#include <stdio.h>
#include <math.h>
int main() {
    int n, t, d = 0, r; double sum = 0;
    printf("Enter number: "); scanf("%d", &n);
    for (t = n; t > 0; t /= 10) d++;    /* count digits */
    for (t = n; t > 0; t /= 10) {
        r = t % 10;
        sum += pow(r, d);
    }
    printf((int)sum == n ? "Armstrong" : "Not Armstrong");
    return 0;
}
/* 153 -> 1^3+5^3+3^3 = 153 -> Armstrong */
```

9.4 Output Prediction with Reasons

[2018–19, 2.5+2.5]

(i) `i=2, j=2; while(--i && j++) printf("%d%d", i, j);`

Ans. Output: 13.

Pass 1: `--i → 1` (true); `j++ → returns 2` (true), `j` becomes 3 $\Rightarrow$ body prints `i=1, j=3 → "13"`.

Pass 2: `--i → 0` (false) $\Rightarrow$ `&&` short-circuits (`j++` not evaluated), loop ends.

(ii) `float me = 2.1; double you = 2.1; if (me == you) ...`

Ans. Output: **Hi.**

2.1 has no exact binary representation. The `float` stores it with 7-digit precision, the `double` with 15-digit precision, so after promotion of `me` to `double` the two values differ slightly $\Rightarrow$ the equality test fails. *Moral: never compare floating-point numbers with `==`.*

10 Last-Minute Revision Sheet

- **Famous constants:** `EOF = -1`; `'a' = 97`, `'A' = 65`, `'0' = 48`; default function return type = `int`.
- **Declarations:** `int *f()` = function returning pointer; `int (*f)()` = pointer to function.
- `switch` works on `int/char/enum` only — never `float/double`; don't forget `break`.
- **Operators:** `^` = XOR (no power operator!); `n<<s =  $n \times 2^s$` ; `n>>s =  $n/2^s$` .
- **Loops:** `while` = entry-controlled (0+ times); `do-while` = exit-controlled (1+ times); semicolon after `for(...)` = empty body trap.
- `break` exits the loop; `continue` skips to next iteration.
- **2-D address:** Row-major $B + w(iN_c + j)$; Column-major $B + w(jN_r + i)$.
- **Complexities:** Bubble/Selection/Insertion $O(n^2)$; Merge sort $\Theta(n \log n)$ from $T(n) = 2T(n/2) + O(n)$; Linear search $O(n)$; Binary search $O(\log n)$. Cards = insertion sort; fewest swaps = selection sort.
- **Asymptotic:** O = upper bound, Ω = lower, Θ = tight.
- **malloc vs calloc:** 1 arg vs 2 args; garbage vs zero-initialised. Free with `free()`.
- **Recursion** uses the *stack*; must have a base case. `gcd(a,b)=gcd(b,a%b)`, base `b==0`.
- **struct** = sum of member sizes (+padding); **union** = size of largest member; type definition alone allocates *no* memory.
- **String** = char array ending in `'\0'`; "HELLO" takes 6 bytes.
- **Pointer arithmetic:** `p+1` advances by `sizeof(type)`; pointer difference = element count.
- **Never** compare floats with `==`; `i = i++` is undefined behaviour.
- **2's complement subtraction:** invert + 1, add, discard end carry.